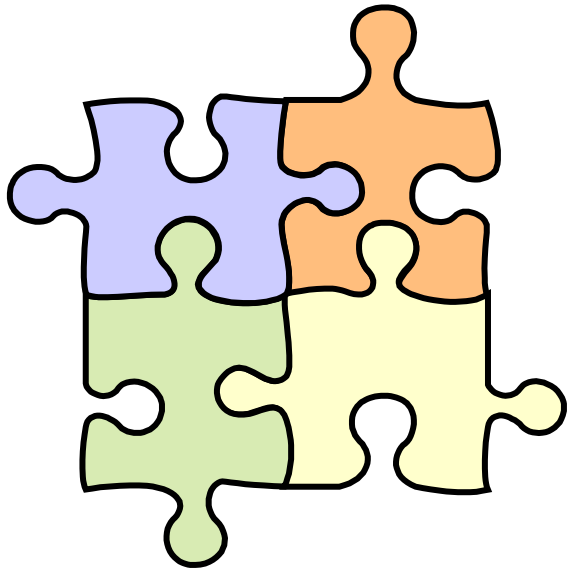
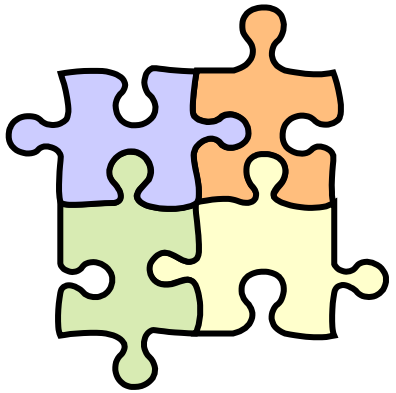
- Lab Info
  - Lab 1
  - Lab 2
  - Lab 3

Link to Lecture Notes





Lab 1

Lab 1 :: Prep

humble

```
sudo apt install ros-$ROS_DISTRO-turtlebot3-gazebo
```

```
export TURTLEBOT3_MODEL=burger
```

```
ros2 launch turtlebot3_gazebo empty_world.launch.py
```

```
# Replace 'humble' with your actual ROS 2 version if different
sudo apt update
sudo apt install ros-humble-turtlebot3
```

```
export TURTLEBOT3_MODEL=burger
ros2 run turtlebot3_teleop teleop_keyboard
```

```
source /opt/ros/humble/setup.bash
```

```
ros-admin@ros-setup:~$ ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

turtlesim ≠ turtlebot3 (sim & real)

Terminal 2: Start Teleop (The Controller)

Run the keyboard control node so you can move the robot around. Keep this window focused to use your keys.

bash

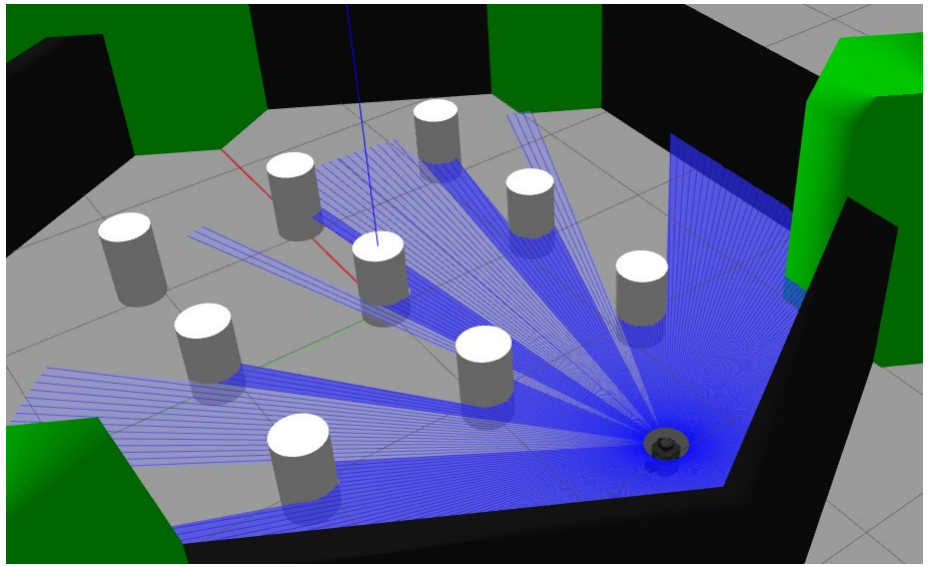
```
export TURTLEBOT3_MODEL=burger
ros2 run turtlebot3_teleop teleop_keyboard
```

Terminal 3: Start RViz2 (The Visualization)

Launch the visualizer to see the sensor data live while you move.

bash

```
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_bringup rviz2.launch.py
```



Lab 1 :: First Steps in the Turtlebot3 Burger Lab ...

There are 4 Turtlebots {red, blue, green, white} with a remote laptop each ... sit down and remember your <Color> ...

- Insert a battery into your turtlebot; switch on the turtlebot; place the turtlebot on a „stand“ on the ground
- Remote laptop login: user **eit**, password **electron**
- Open a terminal and ping your turtlebot, adjust the command to your <Color> :

```
eit@BlueDevelop:~$ ping bluebot
```

  If you receive an answer, all is fine; kill the ping (CTRL-C) ... otherwise call for help.

- Login to the turtlebot via ssh:

```
eit@BlueDevelop:~$ ssh -l obv bluebot
```

  User **obv**, password **turtle01**; if the login works, all is fine ... otherwise call for help. Keep the shell open.
  In this shell you are working on the Single Board Computer (SBC, Raspberry Pi) on the turtlebot.
  Launch the boot script on the turtlebot:

```
obv@bluebot:~$ ros2 launch turtlebot3_bringup robot.launch.py
```

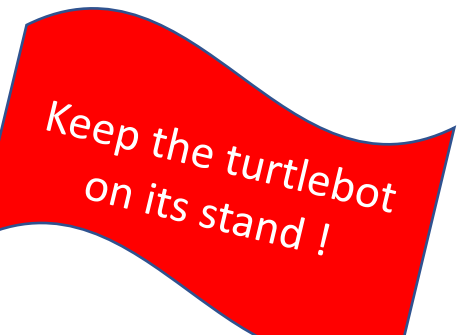
- Open a second shell. Check out the nodes, topics, services:

```
eit@BlueDevelop:~$ ros2 node list
...
```

- Write down the number of nodes, topics, services.

- Start rqt. Move the window to the big screen.
  `eit@BlueDevelop:~$ rqt`
- Open the Message Publisher plugin. Choose the topic /cmd_vel. Set the parameters and publish the message (check box).
  linear.x=0.3, angular.z=0.4 Debug until the robot moves.
- Open a third shell. Observe the published messages via `eit@BlueDevelop:~$ ros2 topic echo /cmd_vel`
- Stop the motion by publishing zeros.
- Try to find values for linear.x and angular.z so that the robot moves „slowly“. Write down the values.

- Open the Service Caller plugin. Choose the Service /sound and call the service. Write down what happens.
- Go to an available shell on BlueDevelop. Find out about the request and response arguments of the /sound service. Write them down.



See the lecture slides for help

- Create a package lab1pkg.
- Create the Python file stop_turtle with a class StopTurtle that creates the node stop_turtle_node.
  - The node shoud publish all zeros to the /cmd_vel topic.
    - Execute the publish(cmd) multiple times with 0.2 secs pause in between: for n in range(1,10) ... sleep(0.2)
  - Develop small increments.
    - Start with a simple logger output that the node was started.
    - To make sure things work: publish a value larger then zero, and only eventually zero.
- Define the executable stop_turtlebot3 in setup.py.
- Test the node by moving the turtlebot forward (via rqt) and stopping it via the new node.

Naming:	
Package	lab1pkg
Source *.py	stop_turtle
Class	StopTurtle
Executable	stop_turtlebot3
Node	stop_turtle_node

Keep the turtlebot on its stand !

See the lecture slides for help

- Create another node in the package lab1pkg.
  - The node shoud move the turtlebot foward and turn using „slow“ random values.
  - Change the values every 2 seconds.
  - Make sure to include the dependencies in package.xml (geometry_msgs, ...) and to update setup.py.
- Test the node while keeping the turtlebot on its stand. Can you stop the bot with your stop_turtlebot3 program?
- If you feel that everything works fine and you can somehow stop the bot before any collision, take the turtlebot off the stand and have it drive around in the lab ...
- Put the turtlebot back on its „stand“.

Naming:

Package	lab1pkg
Source *.py	randwalk_turtle
Class	RandwalkTurtle
Executable	randwalk_turtlebot3
Node	randwalk_turtle_node

- By using the „means of your choice“, learn about the laser scanner. You might
  - review code obtained from the „world brain“ (Internet, AI)
  - play with rqt
  - visualize scanner data via rviz
  - think about a subscriber to laser scanner data
  - experiment with „blocking the view“ of the sensor in various distances
- Do not look straight into the sensor (although it is eye safe).
- Do not spend too much time on this task ☺

Use the appropriate QoS profile:

```
qos = rclpy.qos.QoSProfile(...BEST_EFFORT, ... KEEP_LAST..., depth=1)

create_subscription(LaserScan, "/scan ", self.scan_callback, qos)
```

See the lecture slides for help

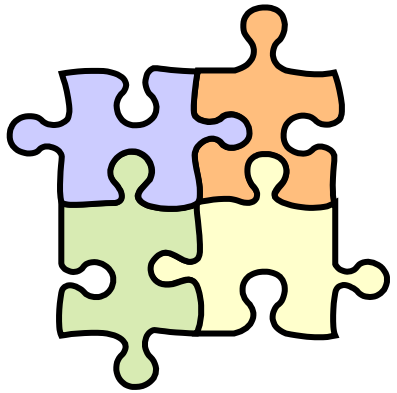
- Create another node in the package lab1pkg.
  - The node shoud move the turtlebot foward and turn as done in RandwalkTurtle(), but now use the laser scanner data to prevent collisions.
  - Think about a reasonable control strategy in case another object is too close to the turtlebot.
- Test the node while keeping the turtlebot on its stand. Use your foot or something else to check out the collision detection and avoidance strategy.
- If you feel that everything works fine, take the turtlebot off the stand and have it drive around in the lab ...
- If you are bored and you do not want to go home: Call the /sound service whenever another object is too close.
- Put the turtlebot back on its „stand“.

Naming:

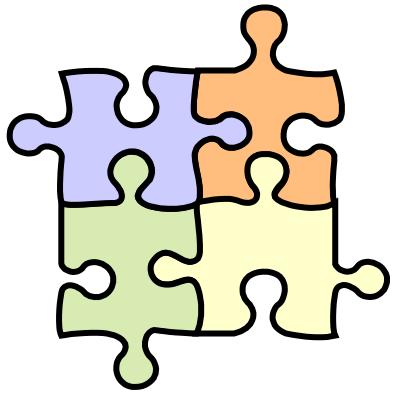
Package	lab1pkg
Source *.py	randwalk_safe_turtle
Class	RandwalkSafeTurtle
Executable	randwalk_safe_turtlebot3
Node	randwalk_safe_ turtle_node

Lab 1 :: Questions :: Answer in Writing

- When you publish a message or call a service, why does only your turtlebot respond and not any other?
- Is using the laser scanner a reliable strategy to avoid collisions?
- What is the detection range of the laser scanner?
- How is the index of the laser scan data mapped to the turtlebot coordinates?
- What are the other components of the /cmd_vel message good for?
- Explain BEST_EFFORT and KEEP_LAST in the context of QoS.
- Visualize the timing of your program. This should explain when new sensor data is processed, when the bot stops, and when it gets a new /cmd_vel.
- ...



Lab 2



Lab 3